Data Preprocessing

Aim:

To preprocess a dataset by handling missing values in numerical and categorical columns to ensure data consistency for further analysis.

Procedure:

1. Import the required libraries — **pandas** and **numpy** for data manipulation.
2. Load the dataset using pd.read_csv() to create a DataFrame.
3. Display the dataset and use df.info() to check data types and missing values.
4. Identify columns with missing data such as *Age* and *Salary*.
5. Replace missing categorical values (e.g., *Country*) with the **mode** (most frequent value).
6. Replace missing numerical values (*Age*, *Salary*) with their respective **mean** values using fillna().
7. Round the filled numerical values to the nearest integer using round().
8. Convert the *Age* and *Salary* columns to integer data types for uniformity.
9. Display the updated DataFrame to confirm that missing values are handled correctly.
10. Save or use the cleaned dataset for subsequent data analysis or machine learning tasks.

```
import pandas as pd
import numpy as np
```

```
df=pd.read_csv('/content/pre_process_datasample - pre_process_datasample (1).csv')
df
```

	Country	Age	Salary	Purchased
0	France	44.0	72000.0	No
1	Spain	27.0	48000.0	Yes
2	Germany	30.0	54000.0	No
3	Spain	38.0	61000.0	No
4	Germany	40.0	NaN	Yes
5	France	35.0	58000.0	Yes
6	Spain	NaN	52000.0	No
7	France	48.0	79000.0	Yes
8	Germany	50.0	83000.0	No
9	France	37.0	67000.0	Yes

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype  
---  ------     --------------  -----  
 0   Country    10 non-null     object 
 1   Age        9 non-null      float64
 2   Salary     9 non-null      float64
 3   Purchased  10 non-null     object 
dtypes: float64(2), object(2)
memory usage: 452.0+ bytes
```

```
df['Country'].fillna(df.Country.mode()[0],inplace=True)
df.Age.fillna(round(df.Age.mean()), inplace=True)
df.Salary.fillna(round(df.Salary.mean()),inplace=True)
```

```
/tmp/ipython-input-898512952.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assign
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting val

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].m


  df['Country'].fillna(df.Country.mode()[0],inplace=True)
/tmp/ipython-input-898512952.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assign
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting val

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].m


  df.Age.fillna(round(df.Age.mean()), inplace=True)
/tmp/ipython-input-898512952.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assign
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting val

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].m


  df.Salary.fillna(round(df.Salary.mean()),inplace=True)
```

```
df.Age = df.Age.round().astype(int)
df.Salary = df.Salary.round().astype(int)
```

```
df
```

	Country	Age	Salary	Purchased
0	France	44	72000	No
1	Spain	27	48000	Yes
2	Germany	30	54000	No
3	Spain	38	61000	No
4	Germany	40	63778	Yes
5	France	35	58000	Yes
6	Spain	39	52000	No
7	France	48	79000	Yes
8	Germany	50	83000	No
9	France	37	67000	Yes

Result

Hence, a Python program was successfully written to preprocess the dataset by imputing missing values with mean and mode, converting data types appropriately, and producing a clean and consistent dataset ready for further processing.